

在第1章《引言》中，我们看到CPU的设计旨在最小化指令执行的延迟，而GPU的设计则旨在最大化执行指令的吞吐量。在第2章《异构数据并行计算》和第3章《多维网格和数据》中，我们学习了用于创建和调用内核以启动和执行线程的CUDA编程接口的核心特性。在接下来的三章中，我们将讨论现代GPU的架构，包括计算架构和内存架构，以及由对这种架构的理解而产生的性能优化技术。本章介绍了GPU计算架构的几个方面，对于CUDA C程序员来说，理解和推理其内核代码的性能行为是至关重要的。我们将首先展示计算架构的高层次、简化的视图，探讨灵活的资源分配、块调度和占用率的概念。然后，我们将深入研究线程调度、延迟容忍、控制分支(control divergence)和同步。最后，我们将通过描述可用于查询GPU中的资源以及在执行内核时帮助估算GPU占用率的工具的API函数来结束本章。在接下来的两章中，我们将介绍GPU内存架构的核心概念和编程注意事项。特别是第5章《内存架构和数据局部性》侧重于芯片上的内存架构，第6章《性能考虑》简要涵盖了芯片外的内存架构，然后详细介绍了GPU架构的各种性能考虑因素。掌握了这些概念的CUDA C程序员能够编写和理解高性能的并行内核。

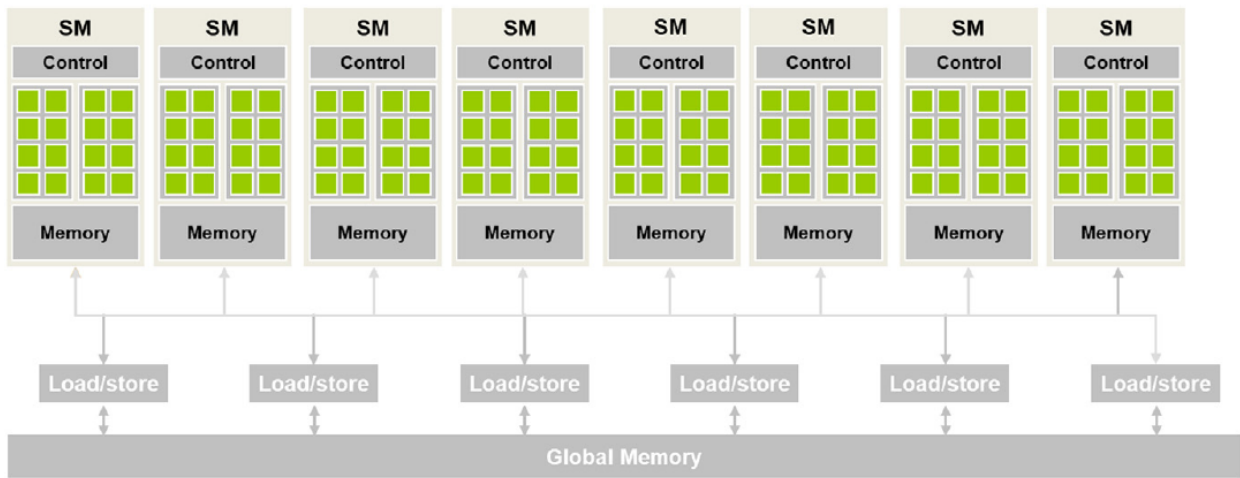


图4.1 有CUDA能力的GPU的架构。

4.1 现代GPU的架构

图4.1展示了一个典型有CUDA能力的GPU的架构的高级视图，供CUDA C程序员参考。它组织成一系列高度线程化的流多处理器 (SM)。每个SM有多个处理单元，称为流处理器或CUDA核心 (以下简称为核心)，如图4.1中SM内显示的小块所示，它们共享控制逻辑和内存资源。例如，Ampere A100 GPU具有108个SM，每个SM有64个核心，总共在整个GPU中有6912个核心。

SM还配备了不同的芯片上内存结构，统称为图4.1中的“Memory”。这些芯片上的内存结构将是第5章《内存架构和数据局部性》的主题。GPU还配备了几GB的芯片外设备内存，称为图4.1中的“全局内存”。尽管较早的GPU使用图形双倍数据速率同步DRAM，但更近期的GPU从NVIDIA的Pascal架构开始可能使用HBM (高带宽内存) 或HBM2，其中包括与GPU紧密集成在同一封装中的DRAM (动态随机存取存储器) 模块。为简便起见，我们将在本书的其余部分广泛地将所有这些类型的内存称为DRAM。我们将在第6章《性能考虑》中讨论访问GPU DRAM的最重要的概念。

4.2 块调度

当调用内核时，CUDA运行时系统启动一个执行内核代码的线程网格。这些线程根据块逐块分配给SM。也就是说，同一块中的所有线程同时分配给同一个SM。图4.2说明了块分配给SM的过程。很可能同时将多个块分配给同一个SM。例如，在图4.2中，每个SM分配了三个块。然而，块需要保留硬件资源进行执行，因此只能同时分配给给定SM的有限数量的块。关于可以同时分配给给定SM的块的数量限制取决于多种因素，这些因素将在第4.6节中讨论。

由于SM的数量有限以及每个SM可以同时分配的块的数量有限，CUDA设备中可以同时执行的块的总数也有限制。大多数网格包含的块数量远远超过这个数字。为确保执行所有块，运行时系统维护一个需要执行的块的列表，并在先前分配的块完成执行时将新块分配给SM。

线程按块逐块分配给SM的方式确保了同一块中的线程同时也在同一个SM上调度。这一保证使得同一块中的线程能够以不同于不同块之间的线程的方式相互交互¹。其中包括栅栏同步，将在第4.3节中讨论。这还包括访问位于SM上的低延迟共享内存，将在第5章《内存架构和数据局部性》中讨论。

注¹：不同线程块中的线程可以通过合作组(Cooperative Groups) API 进行屏障同步。然而，必须遵守一些重要的限制，以确保所有涉及的线程确实在 SM 上同时执行。有兴趣的读者可以查阅 CUDA C 编程指南，了解合作组 API 的正确使用方法。

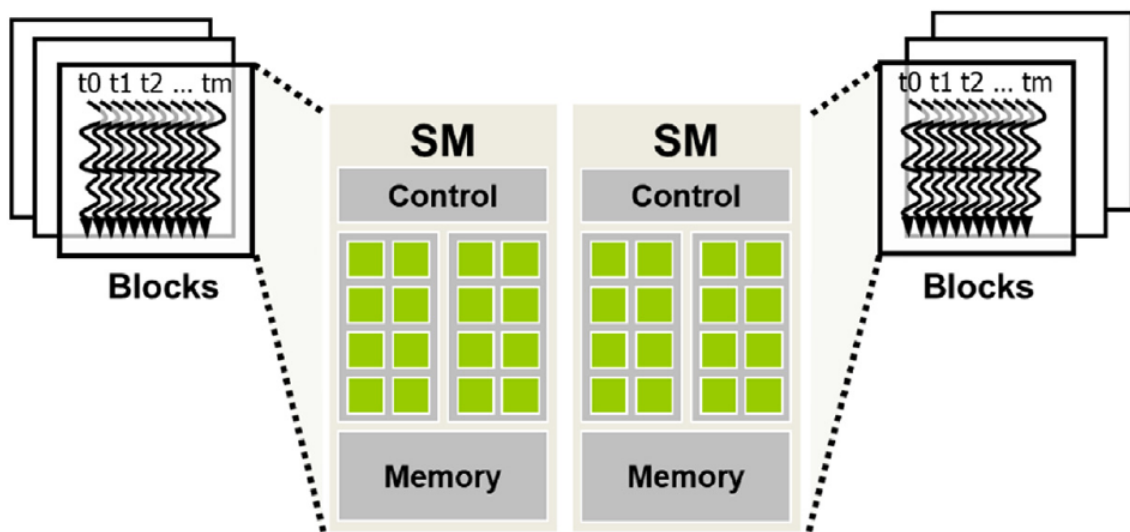


图4.2 将线程块分配给流多处理器 (SMs)。

4.3 同步和透明可扩展性

CUDA 允许同一线程块中的线程使用屏障同步函数 `__syncthreads()` 协调它们的活动。请注意，`__`由两个“_”字符组成。当一个线程调用 `__syncthreads()` 时，它将在调用的程序位置停留，直到同一线程块中的每个线程都到达该位置。这确保了在任何线程继续到下一阶段之前，同一线程块中的所有线程都已完成了它们执行的阶段。

屏障同步是协调并行活动的一种简单而受欢迎的方法。在现实生活中，我们经常使用屏障同步来协调多人的并行活动。例如，假设四个朋友一起去购物中心。他们可以分别去不同的商店购物。这是一种并行活动，比起他们一直作为一个团体依次访问所有感兴趣的商店要高效得多。然而，在离开购物中心之前，需要屏障同步。他们必须等到所有四个朋友都回到车上，然后才能离开。提前完成的人必须等待那些后完成的人。如果没有屏障同步，当车离开时，可能会有一个或多个人被留在购物中心，这可能会严重损害他们的友谊！

图 4.3 说明了屏障同步的执行过程。线程块中有 N 个线程。时间从左到右流逝。一些线程早早到达屏障同步语句，而一些则迟到得多。早到达屏障的线程将等待那些晚到的线程。当最后一个到达屏障时，所有线程都可以继续执行。有了屏障同步，“没有人会被落下”。

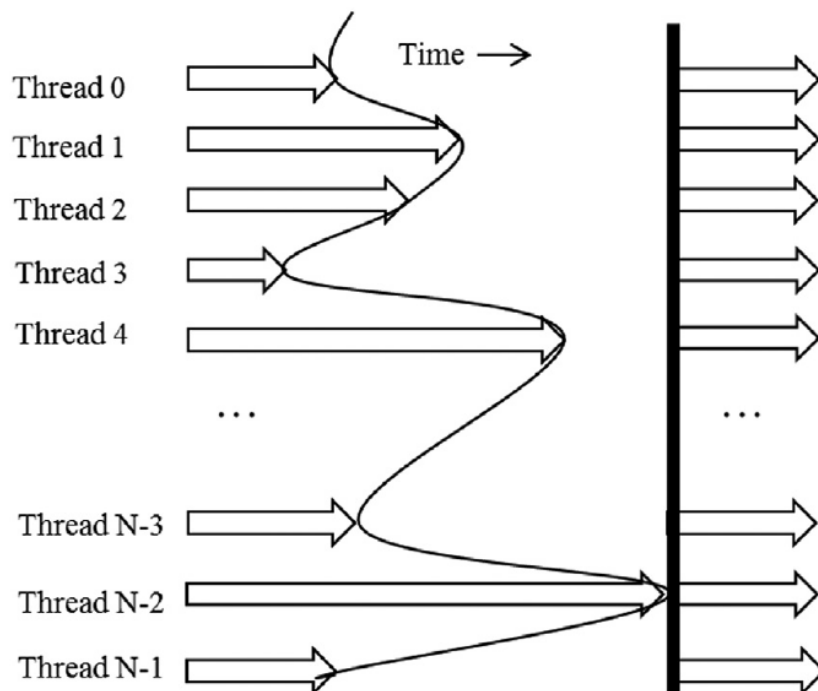


图 4.3 屏障同步的示例执行。箭头表示随时间变化的执行活动。垂直曲线标记了每个线程执行 `__syncthreads` 语句的时间。垂直曲线右侧的空白区域表示每个线程等待所有线程完成的时间。垂直线标记了最后一个线程执行 `__syncthreads` 语句的时间，之后所有线程都被允许继续执行 `__syncthreads` 语句之后的语句。

在CUDA中，如果存在 `__syncthreads()` 语句，则所有线程必须执行该语句。当将 `__syncthreads()` 语句放置在 `if` 语句中时，该块中的所有线程都要执行包含 `__syncthreads()` 的路径，或者一个也不执行。对于 `if-then-else` 语句，如果每个路径都有 `__syncthreads()` 语句，则该块中的所有线程都执行 `then-path` 或者所有线程都执行 `else-path`。两个 `__syncthreads()` 是不同的屏障同步点。例如，在图4.4中，`if` 语句从第04行开始使用了两个 `__syncthreads()`。所有偶数 `threadIdx.x` 值的线程执行 `then-path`，而其余线程执行 `else-path`。在第06行和第10行处的 `__syncthreads()` 调用定义了两个不同的屏障。由于不能保证该块中的所有线程都执行任一屏障，该代码违反了使用 `__syncthreads()` 的规则，将导致未定义的执行行为。一般来说，错误的屏障同步使用可能导致不正确的结果，或者线程永远等待对方，这被称为死锁。程序员有责任避免这种不当使用屏障同步。

```

01 void incorrect_barrier_example(int n) {
02     ...
03     if (threadIdx.x % 2 == 0) {
04         ...
05         __syncthreads{};
06     }
07     else {
08         ...
09         __syncthreads{};
10     }
11 }

```

图 4.4 `__syncthreads()` 函数的错误使用

屏障同步对块内的线程施加了执行约束。这些线程应该在时间上紧密相邻地执行，以避免过长的等待时间。更重要的是，系统需要确保参与屏障同步的所有线程都能访问必要的资源，以最终到达屏障。否则，永远不会到达屏障同步点的线程可能导致死锁。CUDA运行时系统通过将所有块内的线程作为一个单元分配执行资源来满

足这一约束，正如我们在第4.2节中看到的。不仅所有块内的线程必须分配到同一个SM，而且它们需要同时被分配到该SM。也就是说，只有在运行时系统已经获得了该块内所有线程完成执行所需的所有资源时，该块才能开始执行。这确保了块内所有线程的时间接近，并在屏障同步期间防止过度或甚至无限的等待时间。

这引出了CUDA屏障同步设计中的一个重要权衡。通过不允许不同块中的线程进行屏障同步，CUDA运行时系统可以以任意顺序相对于彼此执行块，因为它们中的任何一个都不需要等待其他块。这种灵活性使得可扩展的实现成为可能，如图4.5所示。图中的时间从上到下递进。在成本较低且只有少量执行资源的系统中，可以同时执行少量块，如图4.5左侧所示。在具有更多执行资源的高端实现中，可以同时执行许多块，如图4.5右侧所示。今天的高端GPU可以同时执行数百个块。

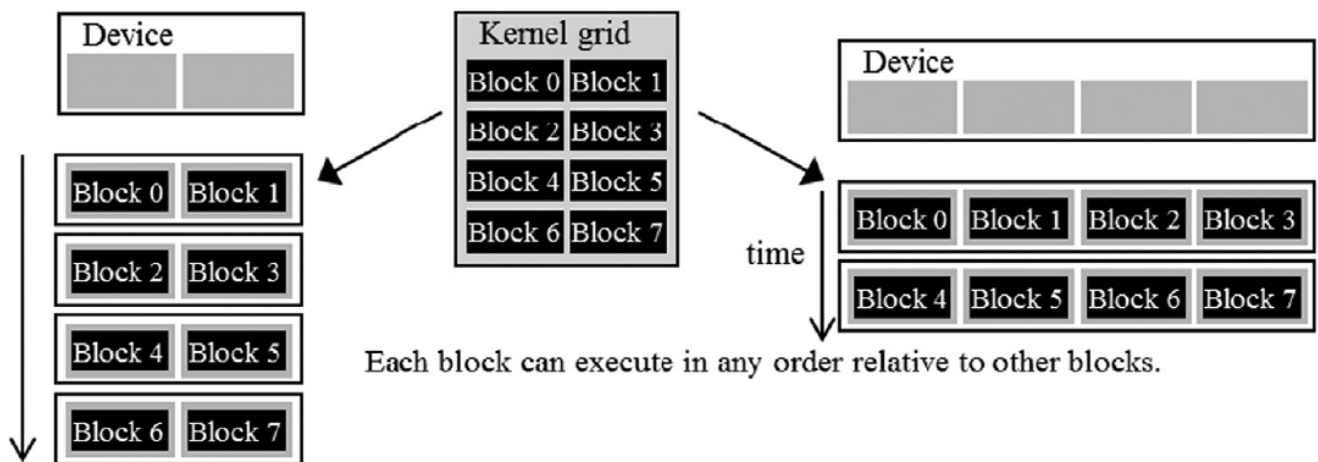


图4.5 块之间不需要同步约束使得CUDA程序具有透明可扩展性。

能够以不同速度执行相同应用代码的能力允许根据不同市场细分的成本、功耗和性能要求生产各种实现。例如，移动处理器可能以极低的功耗执行应用程序，但速度较慢，而台式机处理器可能以更高的速度执行相同的应用程序，但功耗更大。两者都执行相同的应用程序代码，而不需要更改代码。能够在具有不同数量执行资源的不同硬件上执行相同应用程序代码的能力被称为透明可扩展性，这减轻了应用程序开发人员的负担，提高了应用程序的可用性。

4.4 张量和SIMD硬件

我们已经了解到，块可以相对于彼此以任何顺序执行，这使得跨不同设备实现的透明可扩展性成为可能。然而，我们对每个块内部的线程执行时间并没有多少详细说明。在概念上，应该假设块中的线程可以以任何顺序相对于彼此执行。在具有阶段的算法中，应在我们希望确保所有线程在开始下一个阶段之前完成了前一个阶段的执行时使用屏障同步。执行内核的正确性不应取决于任何假设，即某些线程将在不使用屏障同步的情况下同步执行。

CUDA GPU中的线程调度是硬件实现的概念，因此必须在具体硬件实现的背景下讨论。到目前为止，大多数实现中，一旦一个块被分配到一个SM，它会进一步划分为32个线程单元，称为warp。warp的大小是实现特定的，并且在将来的GPU代中可能会有所变化。了解warp可以帮助理解和优化特定CUDA设备上CUDA应用程序的性能。

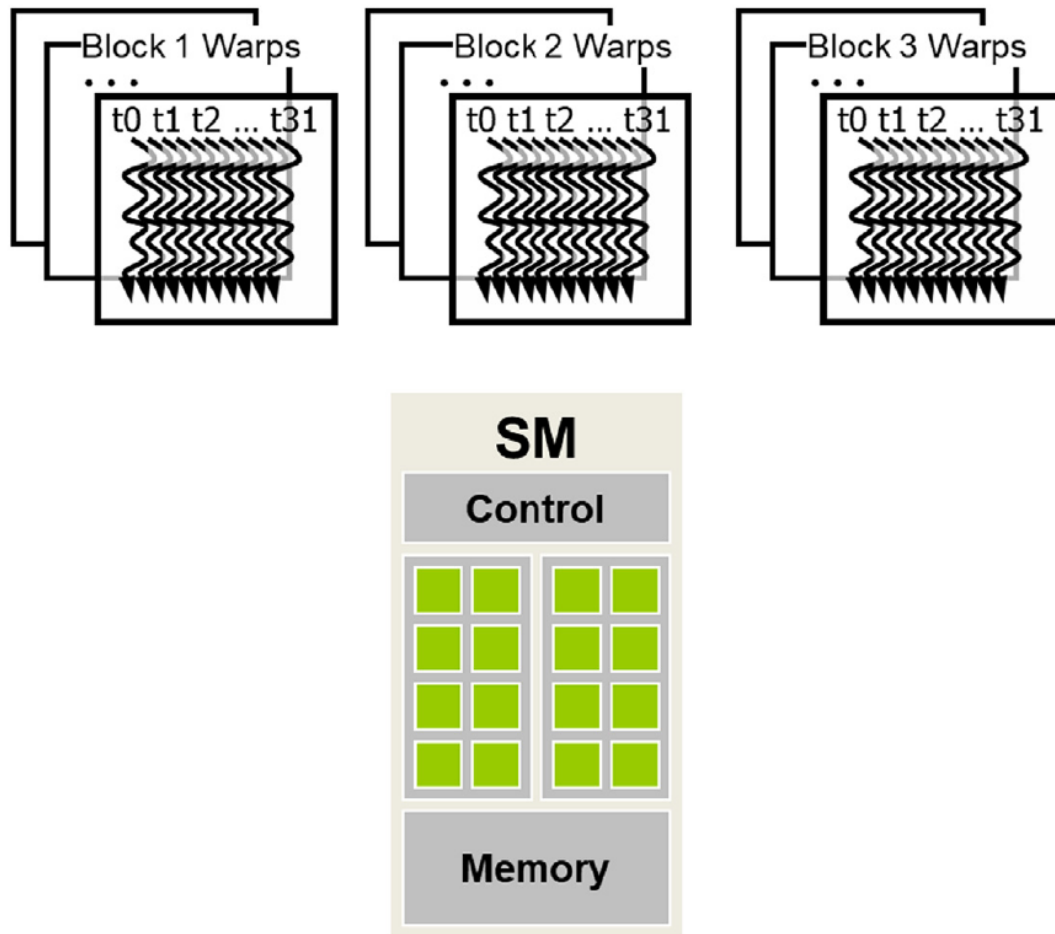


图 4.6 块被划分为warp以进行线程调度。

warp是SM中线程调度的单元。图4.6显示了在一种实现中将块划分为warp的情况。在这个例子中，有三个块——块1、块2和块3——都分配给一个SM。每个块都被进一步划分为warp以进行调度。每个warp由32个连续的threadIdx值组成：线程0到31组成第一个warp，线程32到63组成第二个warp，依此类推。我们可以计算出在给定的块大小和每个SM分配的块数的情况下，驻留在SM中的warp的数量。在这个例子中，如果每个块有256个线程，我们可以确定每个块有 $256/32$ 或8个warp。在SM中有3个块，我们有 $8 * 3 = 24$ 个warp。

块根据线程索引划分为warp。如果一个块被组织成一维数组，即只使用threadIdx.x，划分是直接的。warp中的threadIdx.x值是连续递增的。对于warp大小为32，warp 0以线程0开始，以线程31结束，warp 1以线程32开始，以线程63结束，依此类推。通常，warp n以线程 $32 * n$ 开始，以线程 $32 * (n+1) - 1$ 结束。对于大小不是32的倍数的块，最后一个warp将填充为非活动线程，以填满32个线程位置。例如，如果一个块有48个线程，它将被分成两个warp，第二个warp将填充16个非活动线程。

对于由多个线程维度组成的块，维度将在划分为warp之前被投影为线性化的行主布局。线性布局由将y和z坐标较大的行放在y和z坐标较低的行之后来确定。也就是说，如果一个块由两个维的线程组成，其中一个将通过将所有threadIdx.y为1的线程放在threadIdx.y为0的线程之后来形成线性布局。threadIdx.y为2的线程将在threadIdx.y为1的线程之后放置，依此类推。threadIdx.y值相同的线程按照递增的threadIdx.x顺序连续放置。

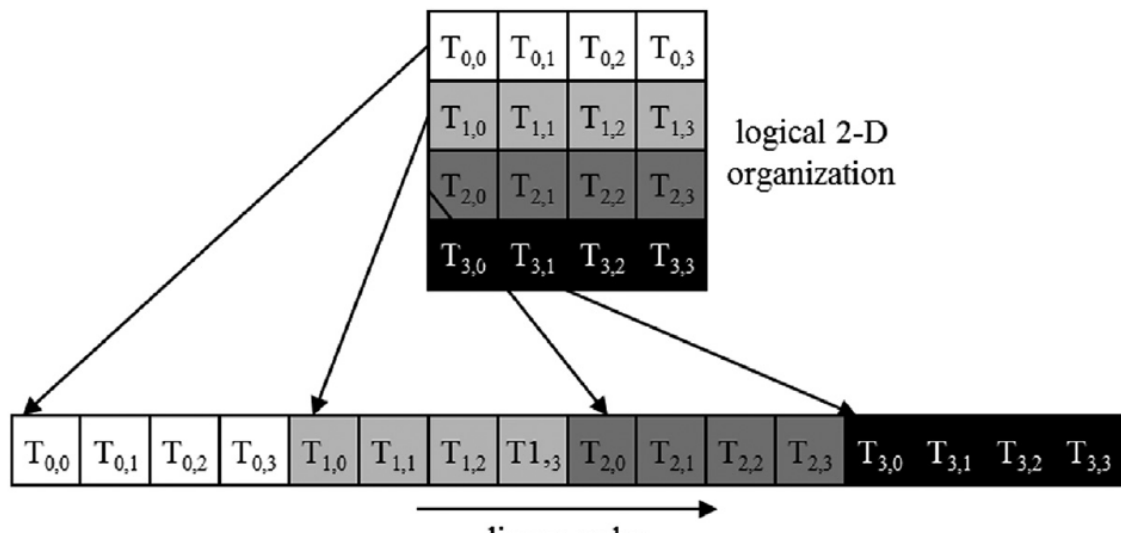


图4.7 将二维线程放入线性布局。

图4.7显示了将二维块的线程放入线性布局的示例。上部显示了块的二维视图。读者应该能够认出与二维数组的行主布局的相似之处。每个线程都显示为 $T_{y,x}$ ，其中 x 为 $threadIdx.x$ ， y 为 $threadIdx.y$ 。图4.7的下半部分显示了块的线性化视图。前四个线程是 $threadIdx.y$ 值为0的线程；它们按照递增的 $threadIdx.x$ 值排序。接下来的四个线程是 $threadIdx.y$ 值为1的线程。它们也按照递增的 $threadIdx.x$ 值放置。在这个例子中，所有16个线程形成半个warp。warp将填充另外16个线程，以完成32个线程的warp。想象一下一个具有 8×8 线程的二维块。这64个线程将形成两个warp。第一个warp从 $T_{0,0}$ 开始，以 $T_{3,7}$ 结束。第二个warp从 $T_{4,0}$ 开始，以 $T_{7,7}$ 结束。读者可以将这个图片画出来作为练习。

对于三维块，我们首先将所有 $threadIdx.z$ 值为0的线程放入线性顺序中。这些线程被视为一个二维块，如图4.7所示。然后将所有 $threadIdx.z$ 值为1的线程放入线性顺序中，依此类推。例如，对于一个三维块（ x 维度中有四个， y 维度中有八个， z 维度中有两个），这64个线程将被划分为两个warp，第一个warp包含 $T_{0,0,0}$ 到 $T_{0,7,3}$ ，第二个warp包含 $T_{1,0,0}$ 到 $T_{1,7,3}$ 。

SM的设计是为了按照单指令多数据（SIMD）模型执行warp中的所有线程。也就是说，在任何时刻，为warp中的所有线程提取和执行一条指令（请参阅“Warps and SIMD Hardware”侧栏）。图4.8显示了SM中的核心如何分组组成处理块，其中每8个核心形成一个处理块，并共享一台指令提取/调度单元。例如，Ampere A100 SM有64个核心，组织成四个每个16个核心的处理块。同一warp中的线程被分配到相同的处理块，该块为warp提取指令，并同时为warp中的所有线程执行它。这些线程将相同的指令应用于数据的不同部分。由于SIMD硬件有效地限制了warp中的所有线程在任何时间点执行相同的指令，warp的执行行为通常被称为单指令，多线程。

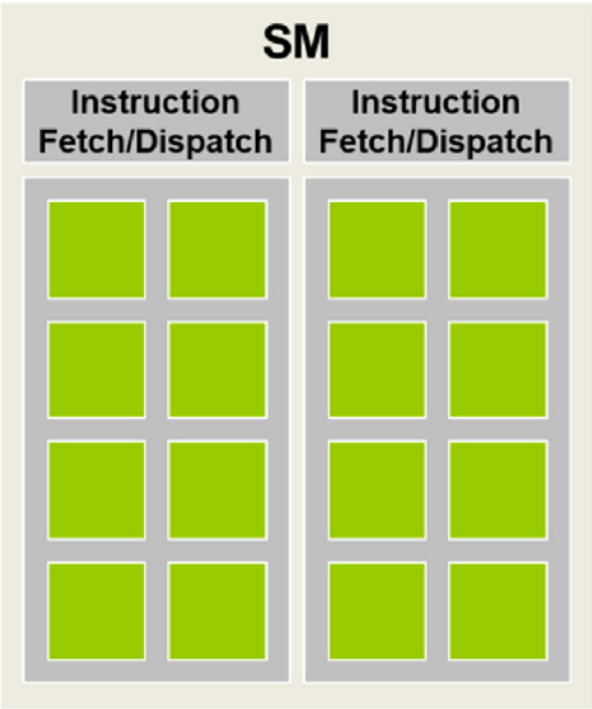


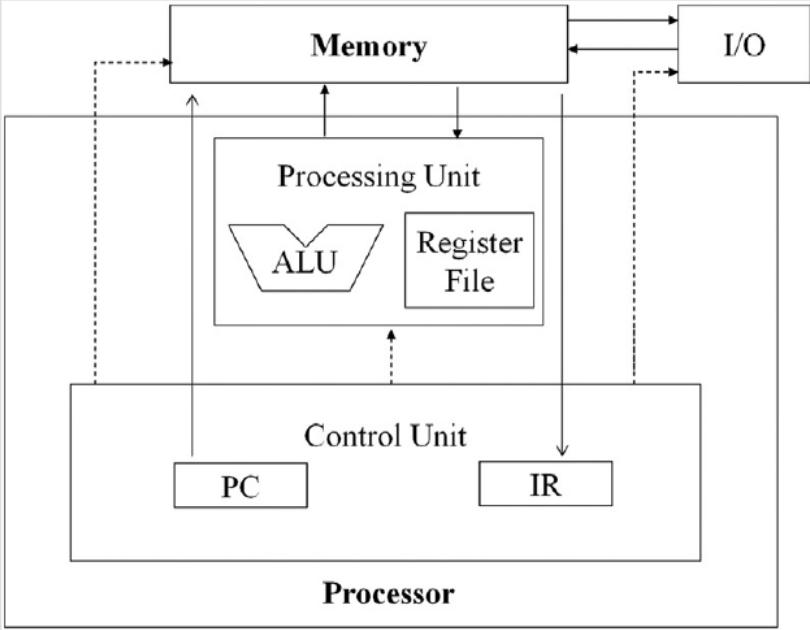
图4.8 流多处理器被组织成用于SIMD执行的处理块。

SIMD的优势在于控制硬件（例如指令提取/调度单元）的成本在许多执行单元之间共享。这种设计选择允许将硬件的较小百分比专用于控制，较大百分比专用于增加算术吞吐量。我们预计在可预见的将来，warp的划分仍将是一种受欢迎的实现技术。然而，warp的大小可以在不同的实现中变化。迄今为止，所有CUDA设备都使用类似的warp配置，其中每个warp由32个线程组成。

Warps and SIMD Hardware

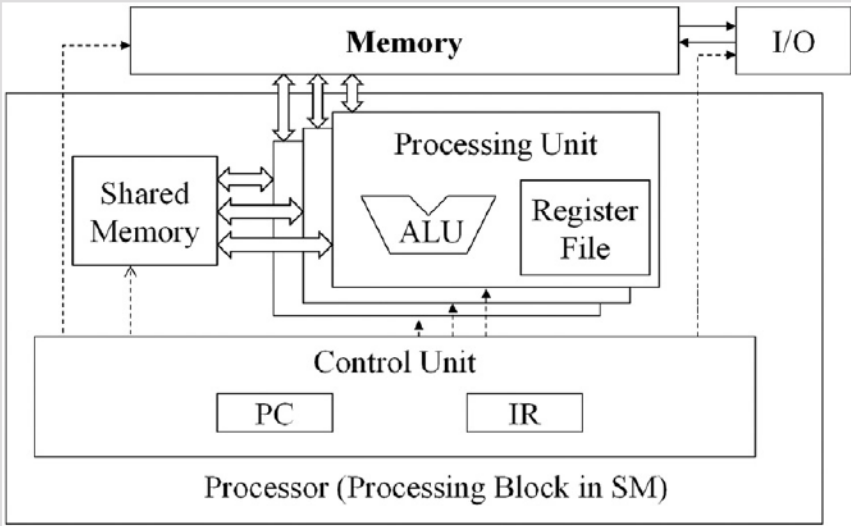
在他1945年的开创性报告中，约翰·冯·诺伊曼描述了一种构建电子计算机的模型，该模型基于先驱性的EDVAC计算机的设计。这个模型，现在通常被称为“冯·诺伊曼模型”，已经成为几乎所有现代计算机的基础蓝图。

冯·诺伊曼模型在下图中进行了说明。计算机具有I/O（输入/输出），允许程序和数据都能够输入到系统中并从系统中生成。为了执行程序，计算机首先将程序及其数据输入到内存中。



程序由一系列指令组成。控制单元维护一个程序计数器（PC），其中包含要执行的下一条指令的内存地址。在每个“指令周期”中，控制单元使用PC将指令提取到指令寄存器（IR）中。然后，检查指令的位以确定计算机的所有组件需要执行的操作。这也是该模型被称为“存储程序”模型的原因，这意味着用户可以通过将不同的程序存储到计算机的内存中来更改计算机的行为。

在以下修改的冯·诺伊曼模型中，以适应GPU设计，说明了将线程作为warp执行的动机。处理器对应于图4.8中的处理块，只有一个控制单元用于提取和分发指令。相同的控制信号（从图4.8中的控制单元到处理单元的箭头）传递到多个处理单元，每个处理单元对应于SM中的一个核心，每个核心执行warp中的一个线程。



由于所有处理单元都由控制单元的指令寄存器（IR）控制，它们的执行差异是由寄存器文件中不同的数据操作数值引起的。这在处理器设计中被称为单指令多数据（SIMD）。例如，尽管所有处理单元（核心）都由一条指令控制，例如add r1, r2, r3，但r2和r3的内容在不同的处理单元中是不同的。

现代处理器的控制单元非常复杂，包括用于提取指令的复杂逻辑和用于指令高速缓存的访问端口。让多个处理单元共享一个控制单元可以显著减少硬件制造成本和功耗。

4.5 控制分支

当warp中的所有线程在处理数据时都遵循相同的执行路径（更正式地称为控制流）时，SIMD执行效果很好。例如，对于if-else结构，当warp中的所有线程执行if-path或全部执行else-path时，执行效果很好。然而，当warp中的线程采用不同的控制流路径时，SIMD硬件将对这些路径进行多次遍历，每个路径一次。例如，对于if-else结构，如果warp中的一些线程遵循if-path而另一些线程遵循else-path，硬件将执行两次。一次执行遵循if-path的线程，另一次执行遵循else-path的线程。在每次遍历期间，遵循另一路径的线程将不被允许产生效果。

当同一warp中的线程遵循不同的执行路径时，我们说这些线程表现出控制分支，即它们在执行中分岔。分支warp执行的多通道方法扩展了SIMD硬件实现CUDA线程的完整语义的能力。虽然硬件对warp中的所有线程执行相同的指令，但它有选择地只让这些线程在对应于它们所采取的路径的通道中产生效果，从而使每个线程都可以似乎采取自己的控制流路径。这保留了线程的独立性，同时利用了SIMD硬件的降低成本。然而，分支的代价是硬件需要执行额外的通道，以允许warp中的不同线程做出自己的决策，以及每个通道中由非活动线程消耗的 execution 资源。

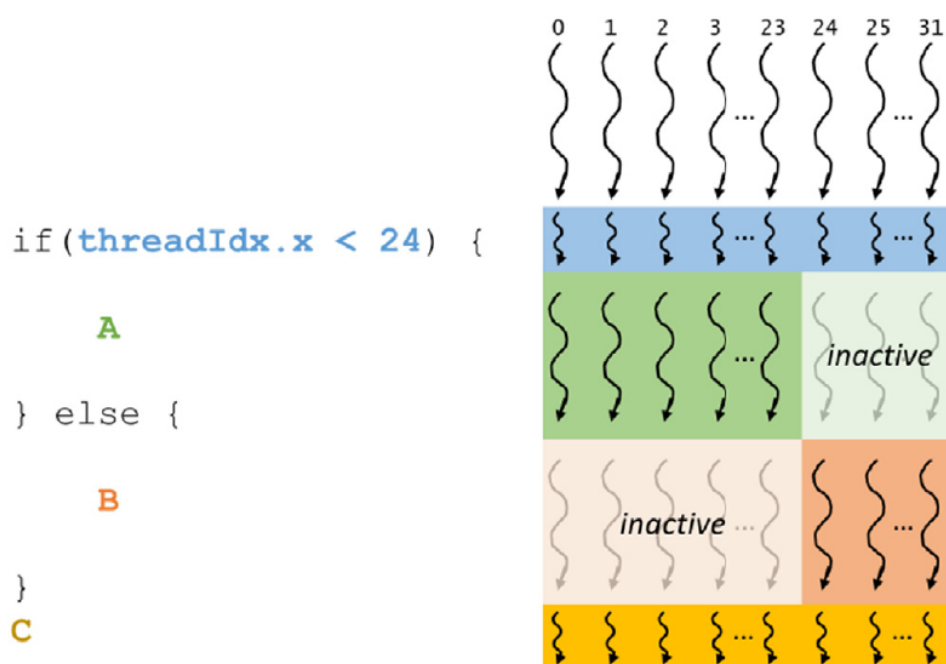


图4.9 warp 在if-else 语句中分支的示例。

图4.9显示了warp如何执行分支的if-else语句的示例。在这个例子中，当由线程0到31组成的warp到达if-else语句时，线程0到23走then-path，而线程24到31走else-path。在这种情况下，warp将通过代码执行一次，其中线程0到23执行A，而线程24到31处于非活动状态。warp还将通过代码执行另一次，其中线程24到31执行B，而线程0到23处于非活动状态。然后，warp中的线程重新汇聚并执行C。在Pascal架构和之前的架构中，这些通道是按顺序执行的，意味着一次通道执行完毕后另一次通道执行。从Volta架构开始，这些通道可以并发执行，意味着一次通道的执行可能与另一次通道的执行交错进行。这个特性被称为独立线程调度。有兴趣的读者可以参考Volta V100架构的白皮书（NVIDIA，2017）了解详细信息。

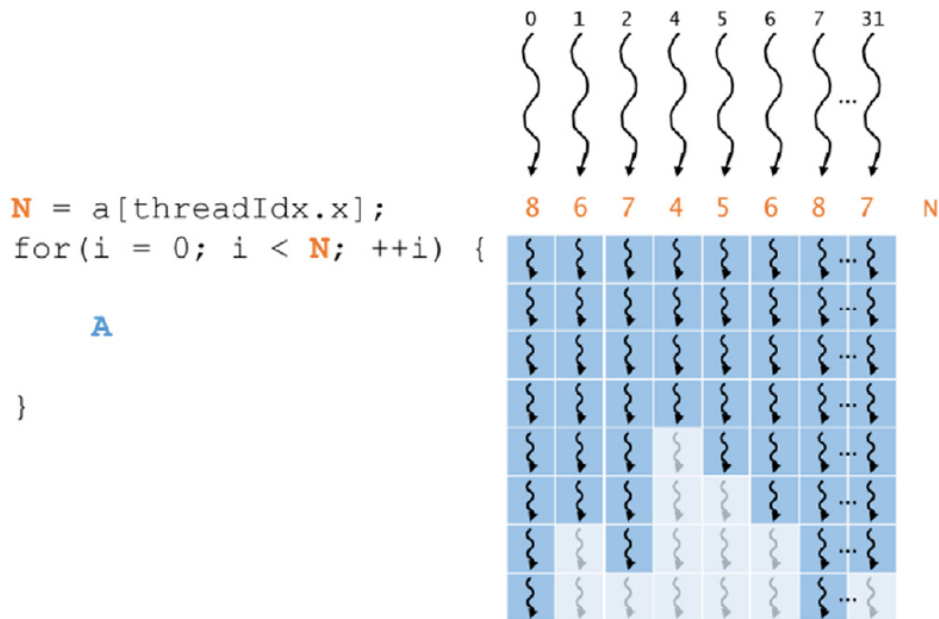


图4.10 warp 在for 循环中分支的示例。

分支也可能在其他控制流结构中出现。图4.10展示了warp如何执行分支的for循环的示例。在这个例子中，每个线程执行不同数量的循环迭代，循环迭代的数量在四和八之间变化。在前四次迭代中，所有线程都是活动的并执行A。在剩余的迭代中，一些线程执行A，而其他线程因为已经完成它们的迭代而不活动。

可以通过检查控制结构的决策条件来确定控制结构是否会导致线程分支。如果决策条件基于threadIdx的值，控制语句可能会导致线程分支。例如，语句if(threadIdx.x > 2) {...}会导致块的第一个warp中的线程遵循两条分支的控制流路径。线程0、1和2会遵循不同于线程3、4、5等的路径。同样，如果循环的循环条件基于线程索引值，循环也可能导致线程分支。

在处理线程映射到数据时，使用具有线程控制分支的控制结构的一个普遍原因是处理边界条件。这通常是因为线程的总数需要是线程块大小的倍数，而数据的大小可以是任意的数字。例如，在第2章的矢量加法kernel中，我们在addVecKernel中有一个if(i < n)语句。这是因为不是所有的矢量长度都可以表示为块大小的倍数。例如，假设矢量长度为1003，我们选择64作为块大小。需要启动16个线程块来处理所有1003个矢量元素。然而，这16个线程块将有1024个线程。我们需要禁用线程块15中的最后21个线程，以防止它们执行原始程序不期望或不允许的工作。请记住，这16个块被分成32个warps。只有最后一个warp (即最后一个块中的第二个warp) 会有控制分支。

请注意，控制分支的性能影响会随着正在处理的矢量大小的增加而减小。对于矢量长度为100的情况，四个warp中的一个会有控制分支，这可能会对性能产生重大影响。对于矢量大小为1000的情况，32个warp中只有一个会有控制分支。也就是说，控制分支只会影响约3%的执行时间。即使它将warp的执行时间加倍，对总执行时间的净影响也将约为3%。显然，如果矢量长度为10,000或更大，313个warp中只有一个会有控制分支。控制分支的影响将远小于1%！

对于二维数据，例如第3章中的颜色到灰度转换示例，if语句也用于处理在数据边缘操作的线程的边界条件。在图3.2中，为了处理62 × 76图像，我们使用了20 = 4 × 5二维块，每个块包含16 × 16个线程。每个块将被分成8个warps；每个warps由一个块的两行组成。总共涉及160个warps (每个块8个warps)。要分析控制分支的影响，请参考图3.5。在区域1的12个块中，没有一个warp会有控制分支。在区域1有96个warps (12 × 8 = 96)。在区域2，所有24个warps都将有控制分支。在区域3，所有底部的warps映射到完全在图像外部的数据。因此，它们都不会通过if条件。读者应该验证，如果图片的垂直尺寸是奇数，这些warps将会有控制分支。在区域4，前7个warps将有控制分支，但最后一个warp将没有。总的来说，160个warps中有31个会有控制分支。

再次强调，控制分支的性能影响会随着水平尺寸中像素数量的增加而减小。例如，如果我们使用 16×16 块处理一个 200×150 的图片，将会有总共130个块（ 13×10 块）或1040个warps。区域1到4中的warps的数量将分别为864个（ $12 \times 9 \times 8$ ）、72个（ 9×8 ）、96个（ 12×8 ）和8个（ 1×8 ）。其中只有80个warps会有控制分支。因此，控制分支的性能影响将小于8%。显然，如果我们处理一个在水平尺寸上有超过1000个像素的真实图片，控制分支的性能影响将小于2%。

控制分支的一个重要含义是不能假设warp中的所有线程具有相同的执行时序。因此，如果warp中的所有线程必须在任何一个线程继续之前完成其执行阶段，必须使用类似于__syncwarp()的屏障同步机制来确保正确性。

4.6 Warp调度和延迟容忍

当线程分配给SM时，通常分配给SM的线程比SM中的核心多。也就是说，每个SM只有足够的执行单元在任何时刻执行分配给它的所有线程的子集。

在早期的GPU设计中，每个SM在任何给定时刻只能执行一个warp的一条指令。在更近期的设计中，每个SM可以在任何时刻执行少量warp的指令。无论哪种情况，硬件只能为SM中所有warp的一个子集执行指令。一个合理的问题是，如果它在任何时刻只能执行它们的一个子集，为什么我们需要给一个SM分配这么多的warp？答案是这是GPU容忍长延迟操作（如全局内存访问）的方式。

当一个warp要执行的指令需要等待先前启动的长延迟操作的结果时，该warp不会被选中执行。相反，将选择另一个不再等待先前指令结果的常驻warp进行执行。如果有多个warp准备执行，将使用优先机制选择一个进行执行。从某些线程的操作的延迟时间中填充其他线程的工作的这种机制通常称为“延迟容忍”或“隐藏延迟”（见“延迟容忍”侧边栏）。

延迟容忍

延迟容忍在许多日常情况下都是必需的。例如，在邮局，每个试图寄送包裹的人在去服务柜台之前理想情况下应该填好所有表格和标签。然而，正如我们都经历过的那样，有些人等待服务台职员告诉他们应该填哪个表格以及如何填写表格。当服务台前排有长队时，最重要的是要最大化服务员的生产力。让一个人在服务员面前填表，而其他人在等待是不明智的做法。服务员应该在这个人填表的同时帮助下一个在排队等待的客户。这些其他客户是“准备好了”，不应该被需要更多时间填写表格的客户阻塞。

这就是为什么一个好的服务员会礼貌地请第一个客户在填写表格时让开，同时服务员为其他客户提供服务。在大多数情况下，第一个客户完成表格并且服务员完成为当前客户提供服务后，他或她将被立即服务，而不是去队伍的末尾。

我们可以将这些邮局客户看作是warp，将服务员视为硬件执行单元。需要填写表格的客户对应于一个依赖于长延迟操作的warp的继续执行。

请注意，warp调度也用于容忍其他类型的操作延迟，如流水线浮点运算和分支指令。

有了足够的warp，硬件很可能在任何时刻找到一个要执行的warp，从而在一些warp的指令等待这些长延迟操作的结果时充分利用执行硬件。准备执行的warp的选择不会在执行时间轴上引入任何空闲或浪费时间，这被称为零开销线程调度（见“线程、上下文切换和零开销调度”侧边栏）。通过warp调度，warp指令的长等待时间被其他warp的指令执行“隐藏”起来。容忍长操作延迟的能力是GPU不像CPU那样将几乎所有芯片面积用于缓存内存和分支预测机制的主要原因。因此，GPU可以将更多的芯片面积用于浮点执行和内存访问通道资源。

线程、上下文切换和零开销调度

基于冯·诺依曼模型，我们准备更深入地了解线程是如何实现的。在现代计算机中，线程是在冯·诺依曼处理器上执行程序的程序和状态。回顾一下，线程包含程序代码、正在执行的代码中的指令以及其变量和数据结构的

值。

在基于冯·诺依曼模型的计算机中，程序的代码存储在内存中。PC跟踪正在执行的程序指令的地址。IR保存正在执行的指令。寄存器和内存保存变量和数据结构的值。

现代处理器被设计为允许上下文切换，其中多个线程可以通过轮流取得进展来共享处理器。通过仔细保存和恢复PC值以及寄存器和内存的内容，我们可以暂停线程的执行并正确地稍后恢复线程的执行。然而，在这些处理器中，在上下文切换期间保存和恢复寄存器内容可能会带来显着的执行时间开销。

零开销调度是指GPU能够使需要等待长延迟指令结果的warp进入休眠状态，并激活一个准备就绪的warp，而不会在处理单元中引入任何额外的空闲周期。传统的CPU会因为从一个线程切换到另一个线程需要将执行状态（如传出线程的寄存器内容）保存到内存并从内存加载传入线程的执行状态而产生这样的空闲周期。GPU SM通过在硬件寄存器中保存所有已分配warp的执行状态来实现零开销调度，因此在从一个warp切换到另一个warp时不需要保存和恢复状态。

为了使延迟容忍有效，希望一个SM分配给它的线程数量要比其执行资源同时支持的线程数量多得多，以最大化在任何时刻找到准备执行的warp的机会。例如，在Ampere A100 GPU中，一个SM有64个核心，但可以同时分配给它最多2048个线程。因此，SM可以同时分配给它的线程数量最多比其核心在任何给定时钟周期支持的数量多32倍。对SM的线程进行过量分配是延迟容忍的关键。当当前执行的warp遇到长延迟操作时，它增加了找到另一个准备执行的warp的机会。

4.7 资源划分和占用

我们已经看到，为了容忍长延迟操作，将许多warp分配给一个SM是可取的。然而，并不总是可能将SM支持的最大数量的warp分配给SM。分配给SM的warp数量与其支持的最大数量之比被称为占用率。要了解阻止SM达到最大占用率的原因，首先要了解SM资源是如何划分的。

SM中的执行资源包括寄存器、共享内存（在第5章“内存体系结构和数据局部性”中讨论）、线程块槽(thread block slots)和线程槽(thread slots)。这些资源在线程之间动态划分，以支持它们的执行。例如，Ampere A100 GPU最多可以支持每个SM 32个块，64个warp（2048个线程）和每个块1024个线程。如果以1024个线程的块大小（最大允许的大小）启动网格，则每个SM中的2048个线程槽将被划分并分配给2个块。在这种情况下，每个SM最多可以容纳2个块。类似地，如果以512、256、128或64个线程的块大小启动网格，2048个线程槽将被划分并分配给4、8、16或32个块。

在块之间动态划分线程槽的能力使得SM变得多才多艺。它们可以执行许多每个具有少量线程的块，也可以执行少量每个具有许多线程的块。这种动态划分与固定划分方法形成对比，固定划分方法中，每个块将收到一定量的资源，而不考虑其实际需求。当块需要的线程少于固定划分支持的线程时，固定划分会导致线程槽浪费，而且无法支持需要更多线程槽的块。

资源之间的限制可能导致资源的低利用率，这会导致微妙的相互作用。这种相互作用可能发生在块槽和线程槽之间。在Ampere A100的例子中，我们看到块大小可以从1024变化到64，分别导致每个SM有2 ~ 32个块。在所有这些情况下，分配给SM的线程总数是2048，这最大化了占用率。然而，请考虑每个块有32个线程的情况。在这种情况下，需要将2048个线程槽划分并分配给64个块。然而，Volta SM一次只能支持32个块槽。这意味着只有1024个线程槽将被利用，即32个块每个32个线程。在这种情况下，占用率为（分配的1024个线程）/（最大2048个线程）= 50%。因此，为了充分利用线程槽并达到最大占用率，每个块至少需要64个线程。

另一种可能对占用率产生负面影响的情况是，每个块的最大线程数不能被块大小整除。在Ampere A100的例子中，我们看到每个SM最多可以支持2048个线程。然而，如果选择块大小为768，SM将只能容纳2个线程块（1536个线程），剩下512个线程槽未使用。在这种情况下，既未达到SM每个块的最大线程数，也未达到SM每个块的最大数量。在这种情况下，占用率为（分配的1536个线程）/（最大2048个线程）= 75%。

前面的讨论没有考虑其他资源约束的影响，例如寄存器和共享内存。在第5章“内存体系结构和数据局部性”中，我们将看到在CUDA内核中声明的自动变量存储在寄存器中。一些内核可能使用许多自动变量，而其他内核可能使用较少的自动变量。因此，应该预期一些内核每个线程需要许多寄存器，而其他内核每个线程需要较少的寄存器。通过在SM中动态划分寄存器，SM可以容纳许多块，如果它们每个线程需要较少的寄存器，以及如果它们每个线程需要更多的寄存器，则需要较少的块。

然而，需要注意寄存器资源限制对占用率的潜在影响。例如，Ampere A100 GPU允许每个SM最多使用65536个寄存器。为了以满占用率运行，每个SM需要足够的寄存器来支持2048个线程，这意味着每个线程不应使用超过 $(65536 \text{ 寄存器}) / (2048 \text{ 线程}) = 32 \text{ 个寄存器}$ 。例如，如果一个内核每个线程使用64个寄存器，那么可以使用65536个寄存器支持的最大线程数为1024个。在这种情况下，无论块大小设置为多少，内核都无法以满占用率运行。相反，占用率最多为50%。在某些情况下，编译器可能执行寄存器溢出以减少每个线程的寄存器需求，从而提高占用率水平。然而，这通常是以线程访问内存中的溢出寄存器值的执行时间增加为代价的，可能会导致网格的总执行时间增加。对共享内存资源在第5章“内存体系结构和数据局部性”中进行了类似的分析。

假设程序员实现了一个内核，每个线程使用31个寄存器，并将其配置为每个块512个线程。在这种情况下，SM将同时运行 $(2048 \text{ 个线程}) / (512 \text{ 个线程/块}) = 4 \text{ 个块}$ 。这些线程将使用 $(2048 \text{ 个线程}) * (31 \text{ 个寄存器/线程}) = 63,488 \text{ 个寄存器}$ ，低于65536个寄存器的限制。现在假设程序员在内核中声明了另外两个自动变量，将每个线程使用的寄存器数增加到33。现在2048个线程所需的寄存器数为67,584个，超过了寄存器限制。CUDA运行时系统可能通过将每个SM仅分配给3个块而不是4个块来处理此情况，从而将所需寄存器数降低到50,688个寄存器。然而，这会将SM上运行的线程数量从2048减少到1536；即通过使用两个额外的自动变量，程序看到了占用率从100%降至75%的减少。这有时被称为“性能悬崖”，即资源使用的轻微增加可能导致并行性和性能显著减少（Ryoo等人，2008）。

读者应该清楚的是，所有动态划分的资源的约束以复杂的方式相互作用。准确确定每个SM中运行的线程数可能是困难的。读者可以参考CUDA占用率计算器（CUDA占用率计算器，网上下载，），这是一个可下载的电子表格，根据内核对资源的使用，计算给定设备实现上每个SM上实际运行的线程数。

【译注：请使用<https://docs.nvidia.com/nsight-compute/NsightCompute/index.html#occupancy-calculator>】

4.8 查询设备属性

我们关于SM资源划分的讨论引发了一个重要问题：我们如何找出特定设备可用的资源量？当CUDA应用程序在系统上执行时，如何查找设备中的SM数量以及每个SM可以分配的块和线程数量？相同的问题也适用于其他类型的资源，其中一些我们到目前为止尚未讨论。一般而言，许多现代应用程序被设计为在各种硬件系统上执行。通常需要应用程序查询底层硬件的可用资源和能力，以利用更有能力的系统，同时补偿性能较差的系统（参见“资源和能力查询”边栏）。

资源和能力查询

在日常生活中，我们经常查询环境中的资源和能力。例如，当我们预订酒店时，我们可以查看附带酒店房间的设施。如果房间配有吹风机，我们就不需要自带一个。大多数美国酒店房间都配有吹风机，而其他地区的许多酒店则没有。

一些亚洲和欧洲酒店提供牙膏，甚至牙刷，而大多数美国酒店则没有。许多美国酒店提供洗发水和护发素，而其他大陆的酒店通常只提供洗发水。

如果房间配有微波炉和冰箱，我们可以把晚餐剩余的食物带上，期望第二天吃掉。如果酒店有游泳池，我们可以带上泳衣，在商务会议后跳进去。如果酒店没有游泳池，但有健身房，我们可以带上跑鞋和运动服。一些高档的亚洲酒店甚至提供运动服！

这些酒店的设施是酒店的属性，或资源和能力的一部分。经验丰富的旅行者会查看酒店网站上的属性，选择最符合他们需求的酒店，并更有效地打包。

每个CUDA设备SM中的资源量是设备的计算能力的一部分。一般而言，计算能力水平越高，每个SM中的资源就越多。GPU的计算能力往往会从一代到下一代逐渐增加。Ampere A100 GPU的计算能力为8.0。

在CUDA C中，有一种内置机制，使主机代码能够查询系统中可用设备的属性。CUDA运行时系统（设备驱动程序）具有一个名为`cudaGetDeviceCount`的API函数，该函数返回系统中可用的CUDA设备数量。主机代码可以通过使用以下语句找出可用的CUDA设备数量：

```
int devCount;
cudaGetDeviceCount(&devCount);
```

虽然可能不太明显，但现代PC系统通常具有两个或更多的CUDA设备。这是因为许多PC系统配备了一个或多个“集成”GPU。这些GPU是默认的图形单元，并提供基本的功能和硬件资源，以执行现代窗口化用户界面的最低图形功能。大多数CUDA应用程序在这些集成设备上的性能不会很好。这将是主机代码迭代遍历所有可用设备，查询其资源和能力，并选择那些具有足够资源以满足应用程序性能的设备的设备的原因。

CUDA运行时将系统中所有可用的设备编号从0到`devCount-1`。它提供了一个名为`cudaGetDeviceProperties`的API函数，该函数返回给定数字的设备的属性。例如，我们可以在主机代码中使用以下语句迭代可用设备并查询其属性：

```
cudaDeviceProp devProp;
for(unsigned int i = 0; i < devCount; i++) {
    cudaGetDeviceProperties(&devProp, i);
    // Decide if device has sufficient resources/capabilities
}
```

内置类型`cudaDeviceProp`是一个C结构类型，其字段表示CUDA设备的属性。读者可以参考CUDA C编程指南，了解该类型的所有字段。我们将讨论一些特别与分配执行资源给线程相关的字段。我们假设属性在由`cudaGetDeviceProperties`函数设置的`devProp`变量中返回。如果读者选择使用不同的变量名，显然需要在以下讨论中替换相应的变量名。

如其名称所示，字段`devProp.maxThreadsPerBlock`给出了在查询的设备中允许的块中线程的最大数量。某些设备允许每个块中有多达1024个线程，而其他设备可能允许更少。未来的设备甚至可能允许每个块超过1024个线程。因此，查询可用设备并确定哪些设备将允许应用程序所需的足够数量的块中的线程是一个好主意。

设备中的SM数量在`devProp.multiProcessorCount`中给出。如果应用程序需要许多SM才能达到满意的性能，它绝对应该检查潜在设备的此属性。此外，设备的时钟频率在`devProp.clockRate`中。时钟速率和SM数量的组合给出了设备的最大硬件执行吞吐量的良好指示。

主机代码可以在字段`devProp.maxThreadsDim[0]`（x维度），`devProp.maxThreadsDim[1]`（y维度）和`devProp.maxThreadsDim[2]`（z维度）中找到每个块的每个维度上允许的最大线程数。使用此信息的一个示例是对于自动调整系统，在评估底层硬件的最佳性能块尺寸时设置块维度的范围。类似地，它可以在`devProp.maxGridSize[0]`（x维度），`devProp.maxGridSize[1]`（y维度）和`devProp.maxGridSize[2]`（z维度）中找到网格的每个维度上允许的最大块数。此信息的典型用途是确定网格是否可以具有足够的线程来处理整个数据集，或者是否需要某种迭代方法。

字段`devProp.regsPerBlock`给出了每个SM中可用的寄存器数量。此字段对于确定内核是否可以在特定设备上实现最大占用或是否将受到其寄存器使用的限制非常有用。请注意，字段的名称有点误导性。对于大多数计算能力级别，块可以使用的寄存器的最大数量确实与SM上可用的总寄存器数量相同。但是，对于某些计算能力级别，块可以使用的寄存器的最大数量小于SM上可用的总寄存器数量。

我们还讨论了warp的大小取决于硬件。warp的大小可以从`devProp.warpSize`字段中获得。

`cudaDeviceProp`类型中还有许多其他字段。随着我们介绍它们设计来反映的概念和特性，我们将在整本书中讨论它们。

4.9 总结

GPU被组织成SM，它包含多个核心的处理块，这些核心共享控制逻辑和内存资源。当启动一个网格时，其块以任意顺序分配给SM，从而实现CUDA应用程序的透明可伸缩性。透明可伸缩性伴随着一个限制：不同块中的线程无法相互同步。

线程按块为单位分配给SM进行执行。一旦块分配给了SM，它进一步分为warp。warp中的线程按照SIMD模型执行。如果同一warp中的线程通过采取不同的执行路径而分支，处理块将按照每个线程仅在其所采取的路径对应的通行中处于活动状态的顺序执行这些路径。

一个SM可能分配给它的线程比它可以同时执行的线程多得多。在任何时候，SM只执行其驻留warp的一个小子集的指令。这使得其他warp可以等待长延迟操作，而不减缓庞大数量的处理单元的整体执行吞吐量。分配给SM的线程数与其支持的最大线程数之比称为占用率。SM的占用率越高，它越能有效地隐藏长延迟操作。

每个CUDA设备对每个SM中可用资源的限制可能是不同的。例如，每个CUDA设备都对其SM可以容纳的块数、线程数、寄存器数和其他资源的数量有限制。对于每个内核，这些资源限制中的一个或多个可能成为占用的限制因素。CUDA C提供了在运行时查询GPU中可用资源的能力。